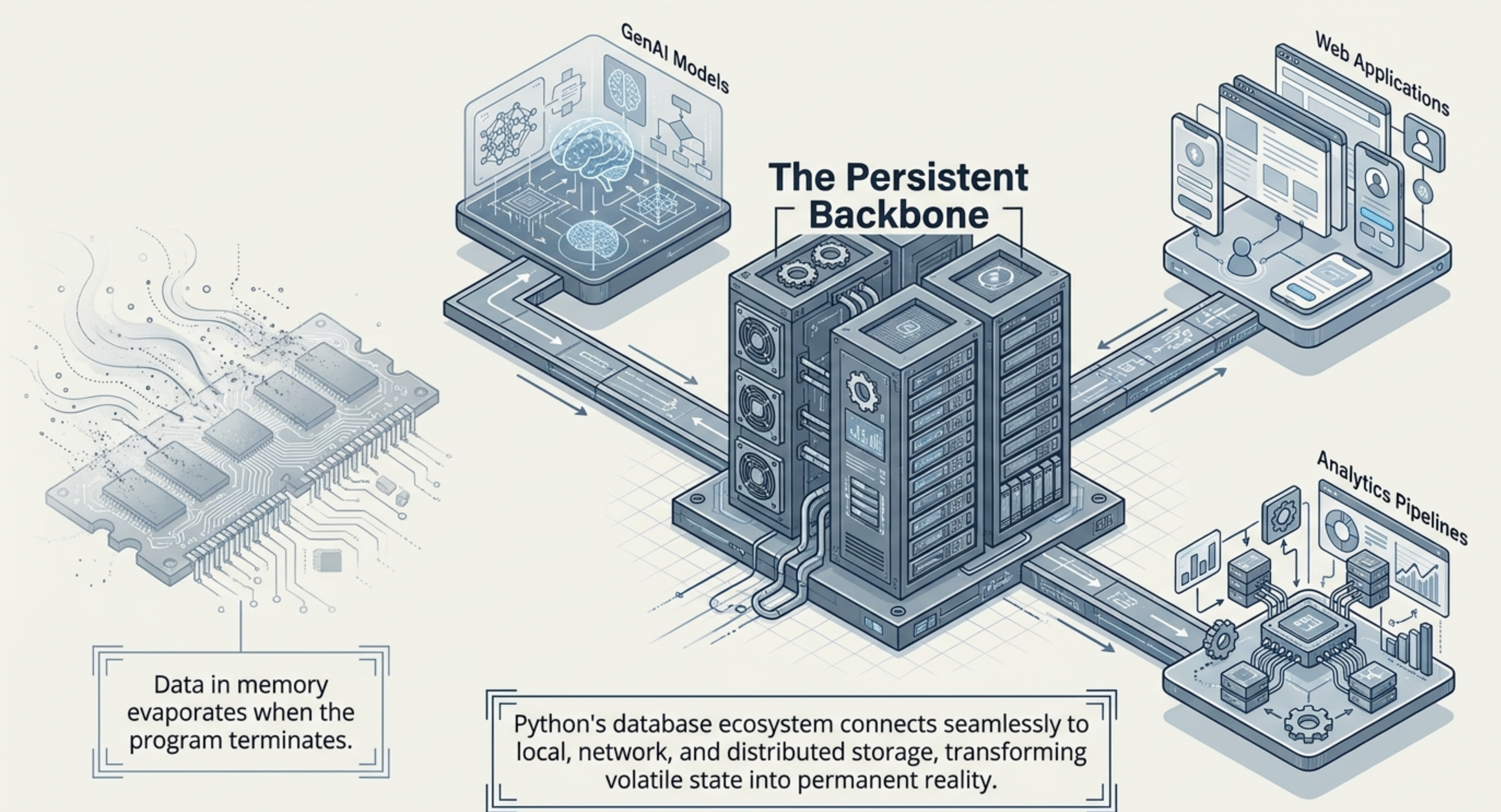
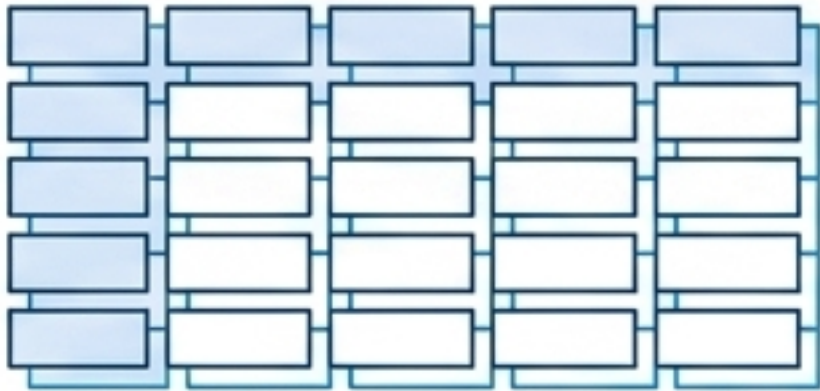
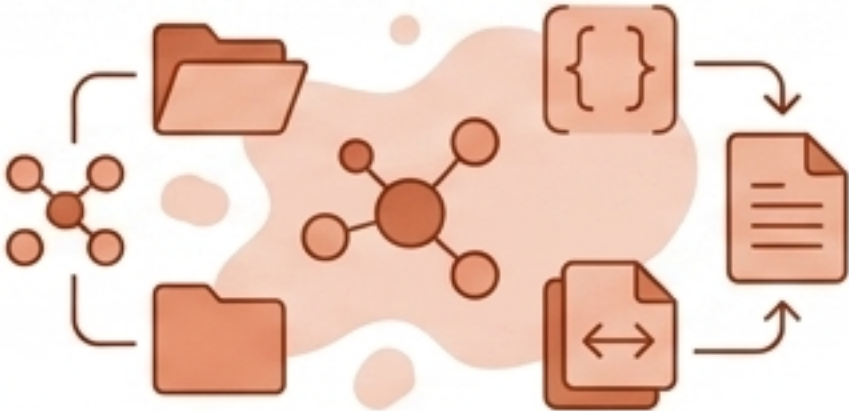


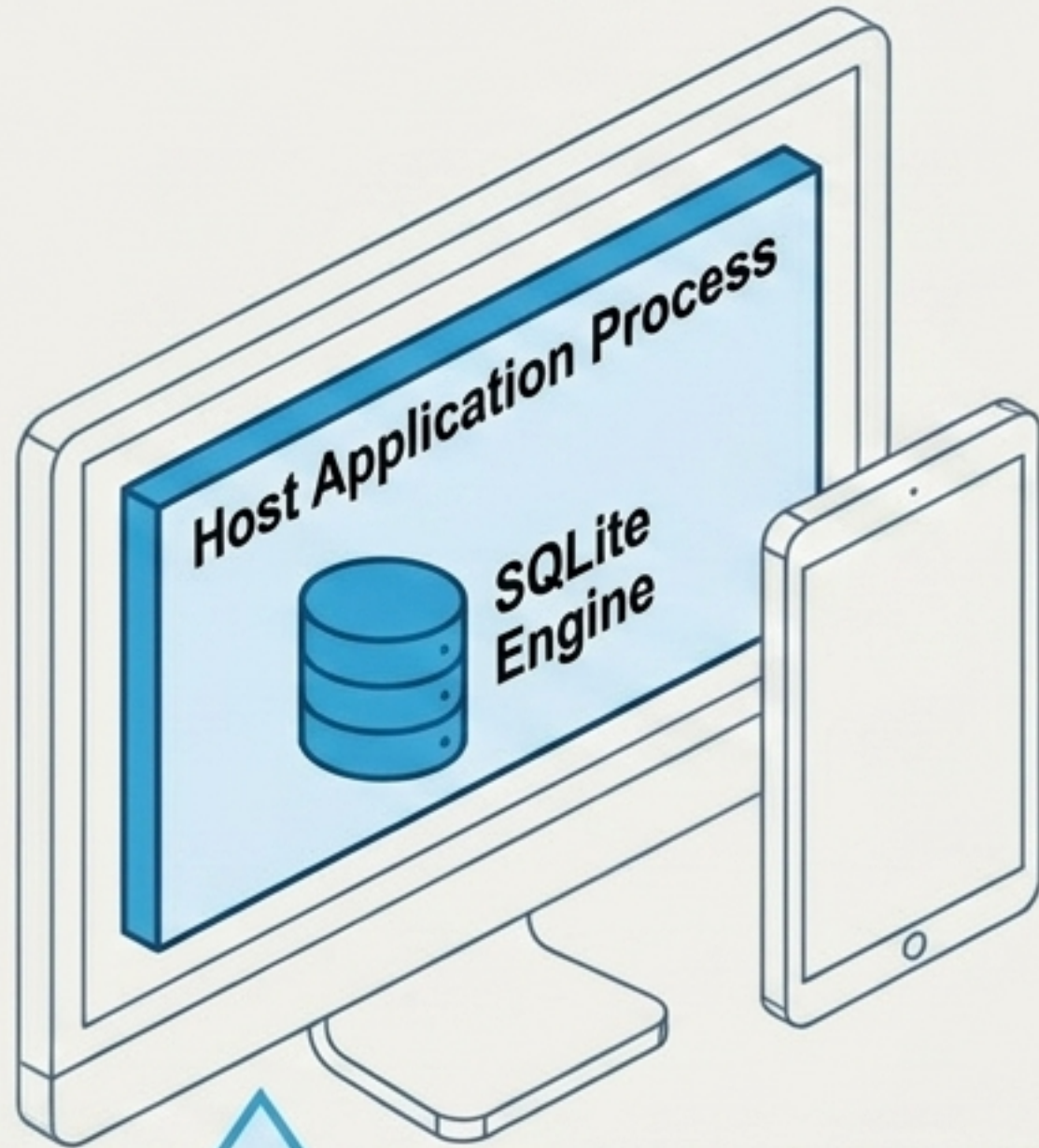


The Architectural Divide: SQL vs NoSQL

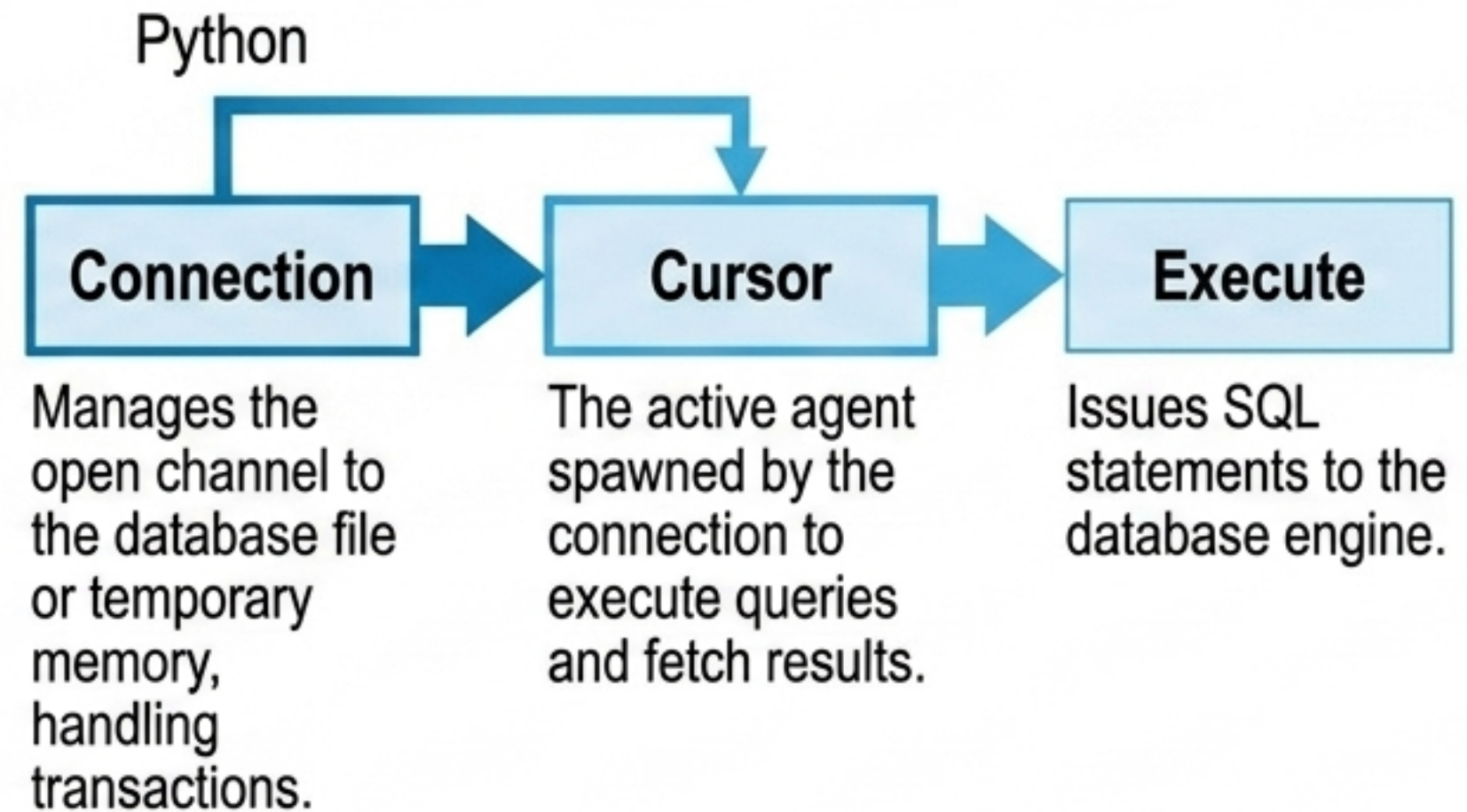
Complementary tools for different architectural topologies.

	SQL (Relational)	NoSQL (Non-Relational)
Data Model	 Rigid predefined tables with rows and columns.	 Flexible documents, graphs, or key-value structures.
Schema & Evolution	Fixed schema; structural changes require explicit migrations.	Schema-flexible; adjacent documents can have completely different structures.
Relationships	First-class support via JOIN operations and Foreign Keys.	Relationships typically handled via nested embedding.
ACID Compliance	Guarantees Atomicity, Consistency, Isolation, and Durability.	Prioritises availability and horizontal partition tolerance.

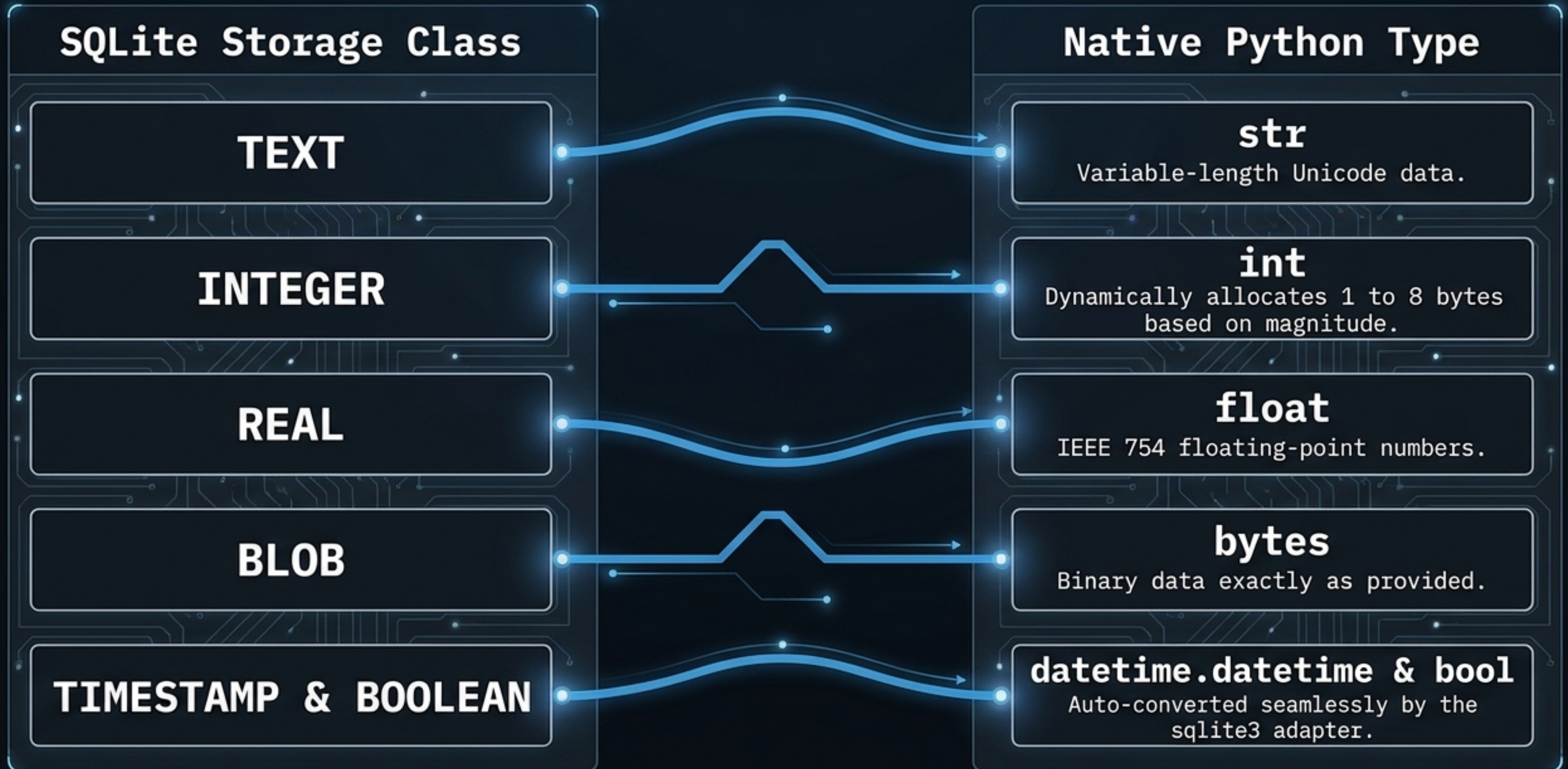
SQLite: The Embedded Relational Engine



Serverless and zero-configuration. The most widely deployed database globally, built natively into Python via the `sqlite3` module.



Bridging SQLite and Native Python Types



Security First: Defeating SQL Injection

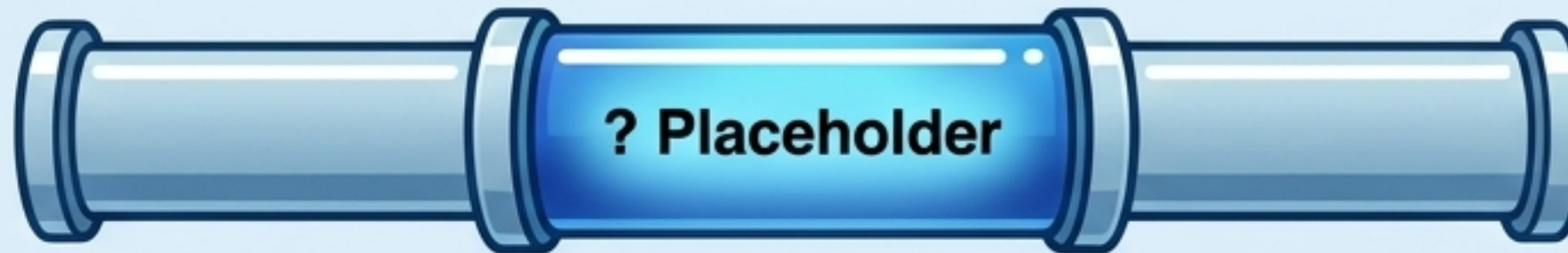
FATAL: String Formatting



```
f'SELECT * FROM users WHERE name = {user_input}'
```

Never build SQL statements using string formatting. Malicious input can alter the structural logic of the database query.

SECURE: Parameterised Queries



```
cursor.execute('SELECT * FROM users WHERE name = ?', (user_input,))
```

Always pass user input as a separate tuple. The database driver isolates the data, making structural alteration impossible.

Security Rule: Never Use String Formatting for SQL. Never build SQL queries using Python string formatting (`f"-strings"`, `%`, `.format()`), it equives structural **a** alteration impossible.

Never build SQL queries using Python string formatting (`f"-strings"`, `%`, `.format()`). Always use parameterized queries with the `?` placeholder. This applies universally — to SQLite, PostgreSQL (which uses `%s` as placeholder), and any other database. This rule is non-negotiable in production code.

Upgrading Results: Row Factories

The Input

`( 'raw_data', 123, 45.6 )`
`( 'raw_data', 123, 45.6 )`
`( 'more_data', 789 )`
`( 'more_data', 789 )`
`( 'example', 0.0 )`
`( 'example', 0.0 )`

Raw Tuples

`row[0], row[1]`

Default queries return plain tuples.
Positional indexing is fragile if
SELECT column orders change.

**sqlite3.Row
Factory**

The Output

Dataclasses

`Coin(id=1,
name='BTC',
price=45000.0)`

`Coin(id=2,
name='ETH',
price=3200.0)`

Dictionaries

`{'coin_id': 1,
'name': 'BTC',
'price': 45000.0}`

`{'coin_id': 2,
'name': 'ETH',
'price': 3200.0}`

Structured Dataclasses & Dictionaries

`row['coin_id']`

Enables dictionary-like named access.
Perfect for IDE autocompletion, static
type checking, and passing data to
Pandas DataFrames.

Scaling Up: Enterprise PostgreSQL

Transitioning from embedded files to networked client-server architectures.

Python Client
Application



TCP/IP Network Protocol



PostgreSQL
Database Cluster



The DB-API 2.0 Advantage

SQLite (sqlite3)

PostgreSQL (psycopg2)

Because both adapters follow the strict PEP 249 specification, the code structure remains identical. Transitioning requires only updating the connection string and changing the placeholder from '?' to '%s'.

Enterprise Features:

- Full ACID Compliance
- Advanced JSONB & B-Tree Indexing
- Strict Numeric Precision
- Multi-user concurrent access

The NoSQL Paradigm: Flexibility & Scale

Breaking free from predefined tables to support evolving schemas and horizontal scale.



Document Stores

Stores data in flexible, schema-less JSON/BSON documents. Ideal for user profiles and ML feature sets.

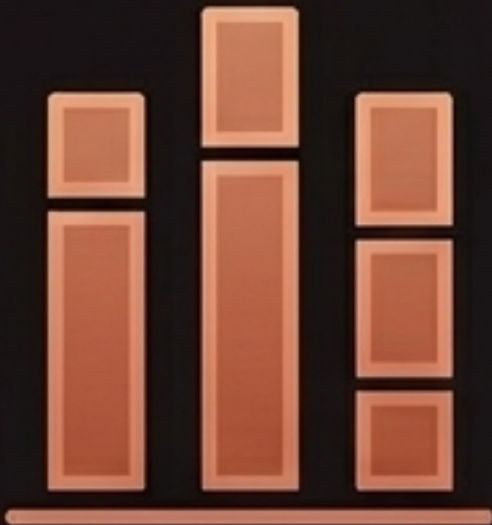
Example: MongoDB



Key-Value Stores

Optimised for blistering speed and temporary state. Ideal for caching and session management.

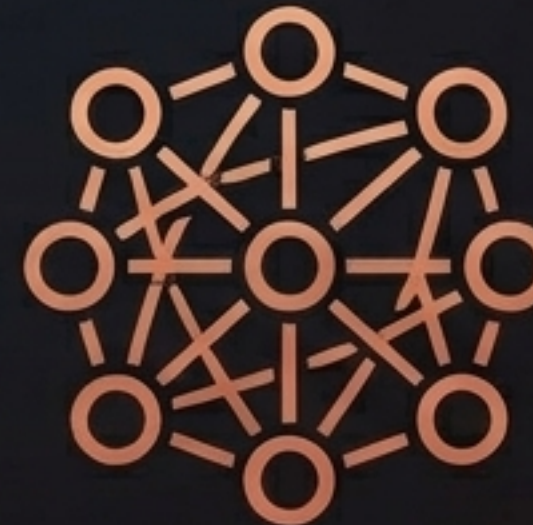
Example: Redis



Column-Oriented

Built for massive horizontal scale. Ideal for time-series data and IoT sensor streams.

Example: Cassandra

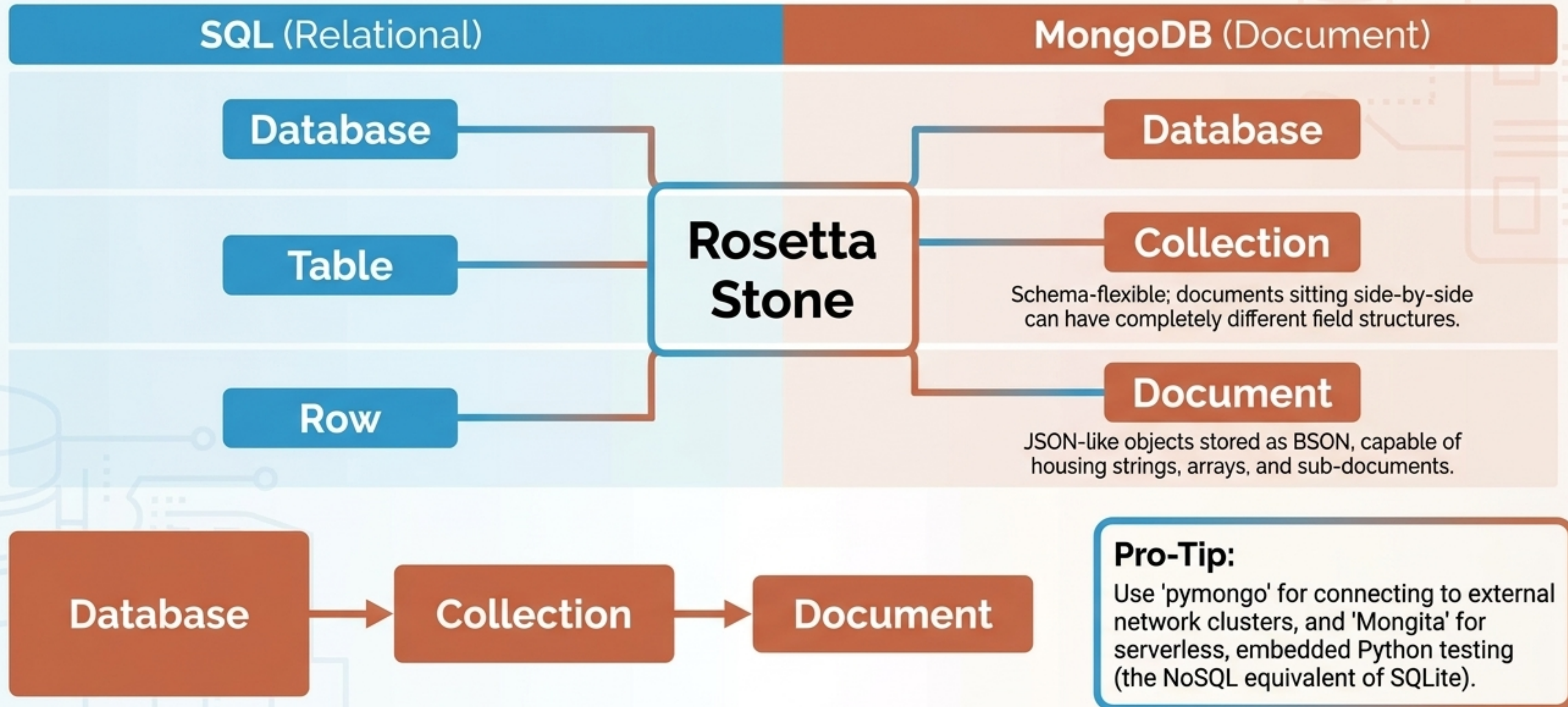


Graph Databases

Specialises in heavily interconnected data. Ideal for recommendation engines and fraud detection.

Example: Neo4j

MongoDB: The Document Store



MongoDB Syntax & Atomic Updates

Bypassing SQL strings entirely by using native Python dictionaries.

```
query = {  
    '$and': [  
        {'status': 'active'},  
        {'score': {'$gt': 85}}  
    ]  
}
```

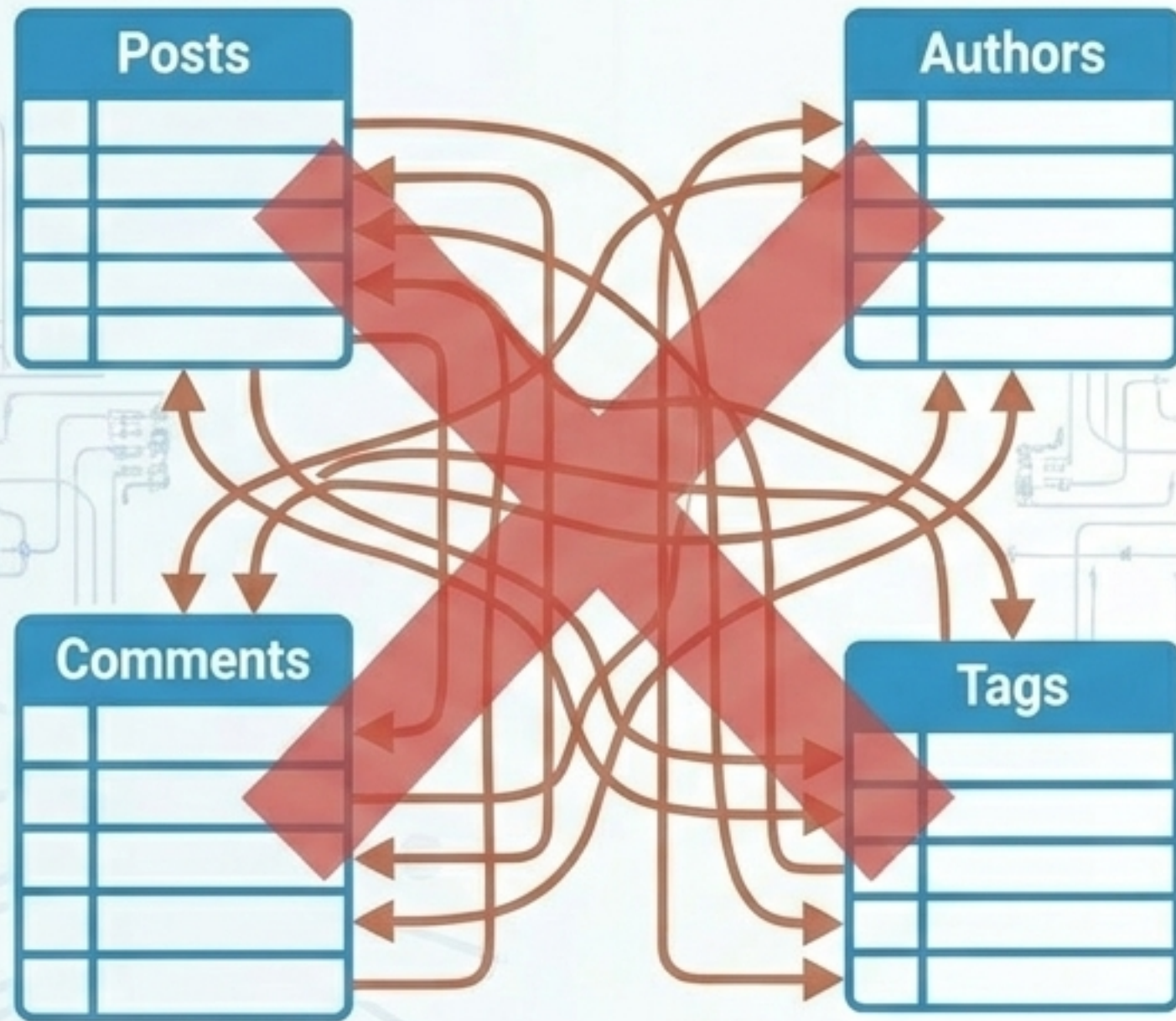
Filter Logic: Utilises explicit operators (greater than, in, and) for complex matching.

Modifier: Applies new values to specific fields, creating them if they do not exist.

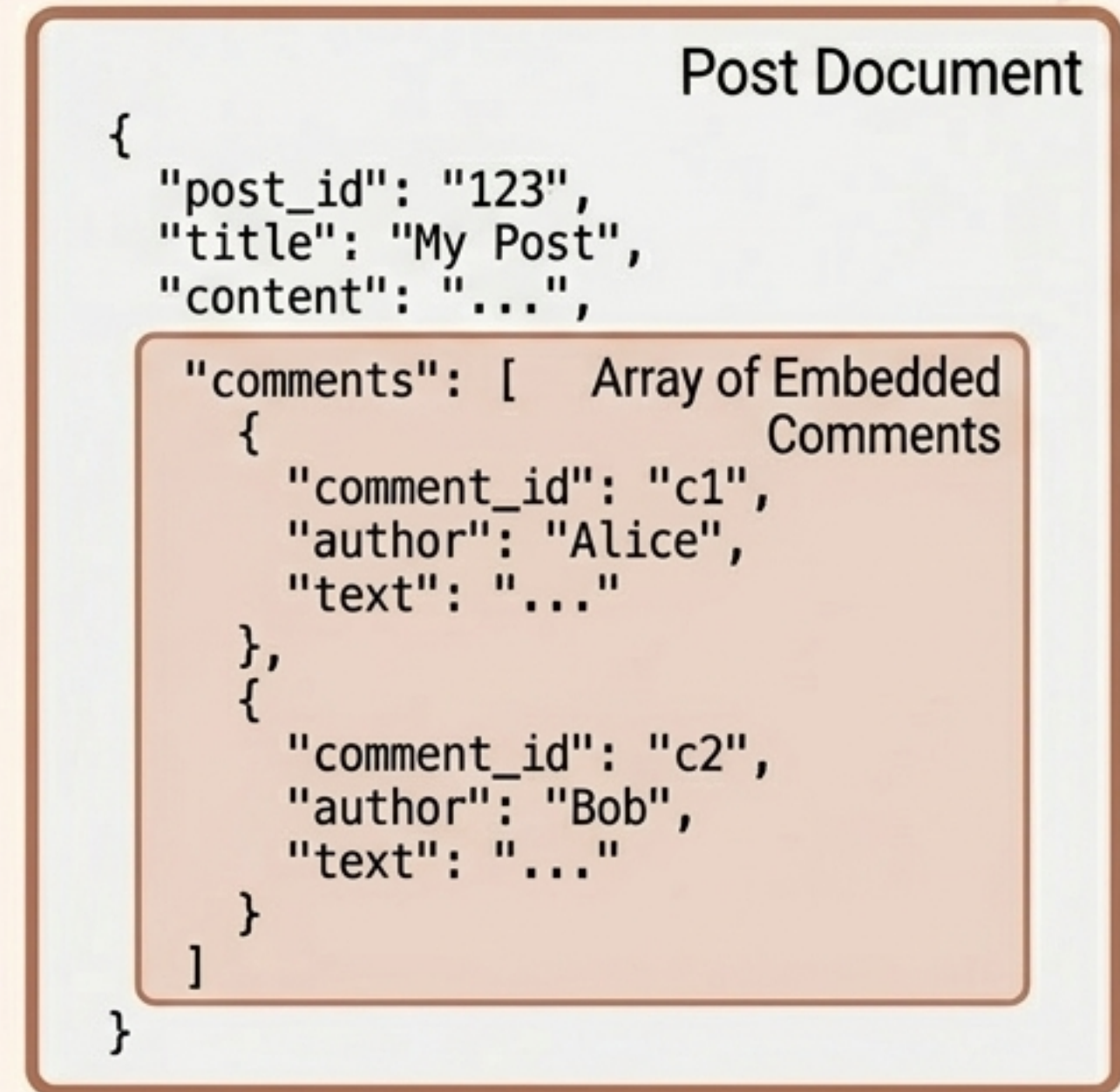
Atomic Operator: Safely increments running totals without race conditions, even during simultaneous modifications.

```
update = {  
    '$set': {'badge': 'expert'},  
    '$inc': {'login_count': 1}  
}
```


Advanced NoSQL: Bypassing the JOIN



Costly JOIN Operations: Relational databases must reconstruct scattered data across multiple tables, increasing read latency.



Hierarchical Embedding: Naturally handles one-to-many relationships by embedding related lists directly inside the parent document. Zero JOINS required.

SQLAlchemy: Object-Relational Mapping

The ultimate abstraction layer: generating underlying SQL from native Python objects.



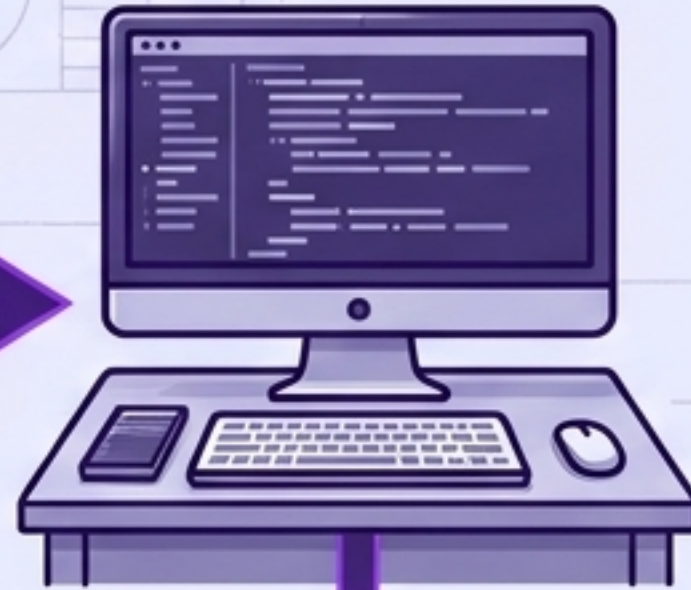
An ORM (Object Relational Mapper) eliminates raw SQL strings. Developers interact strictly with Python objects, enabling database portability (switch from SQLite to Postgres by changing a single URL), static type checking, and robust IDE support.

Engines and Sessions: Unit of Work



The Engine

The connection factory. Manages the database URL and handles connection pooling.



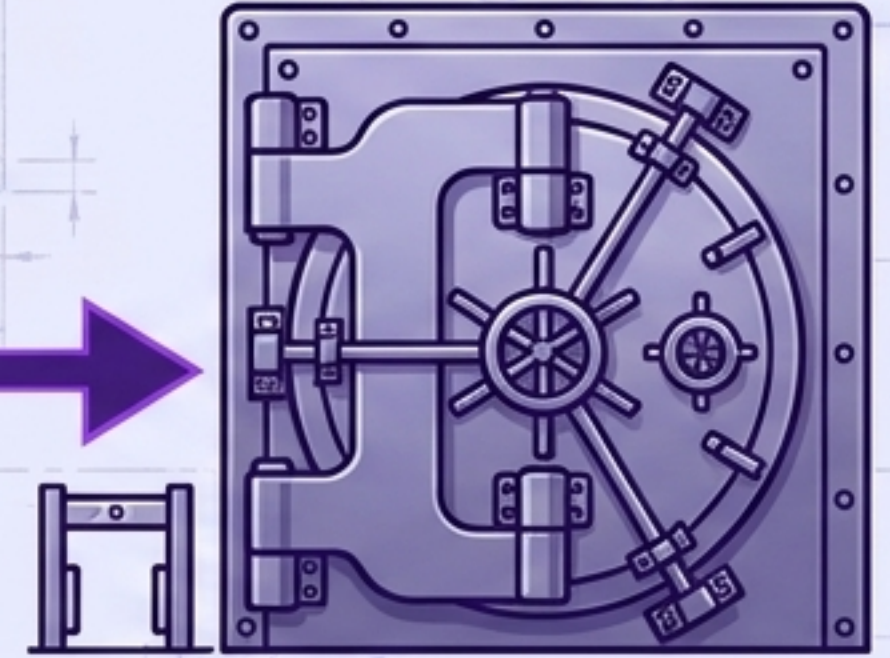
The Session

The primary developer workspace. Acts as a Unit of Work where queries are executed and objects are modified.



Identity Map

Automatically tracks any changes made to loaded Python objects in memory. No manual UPDATE statements necessary.

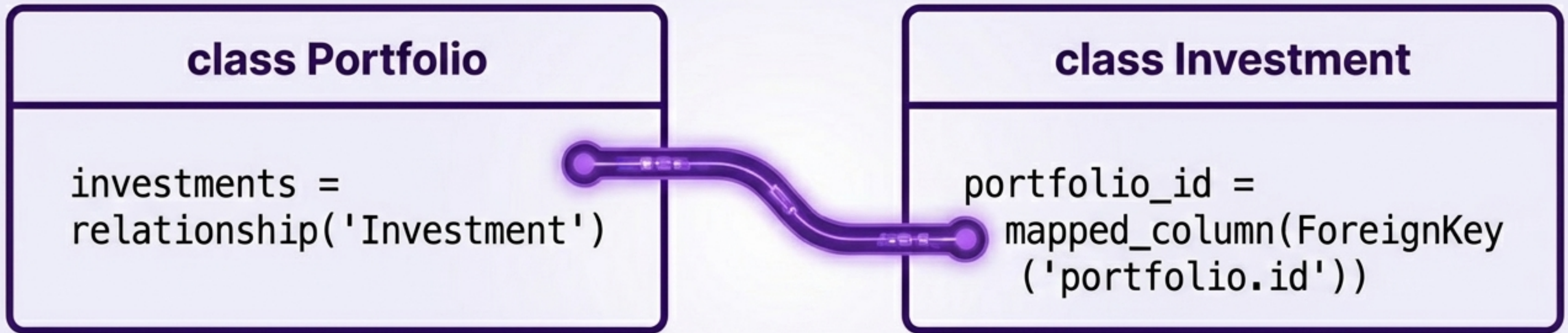


Commit()

Flushes accumulated changes as a single, atomic transaction to the database.

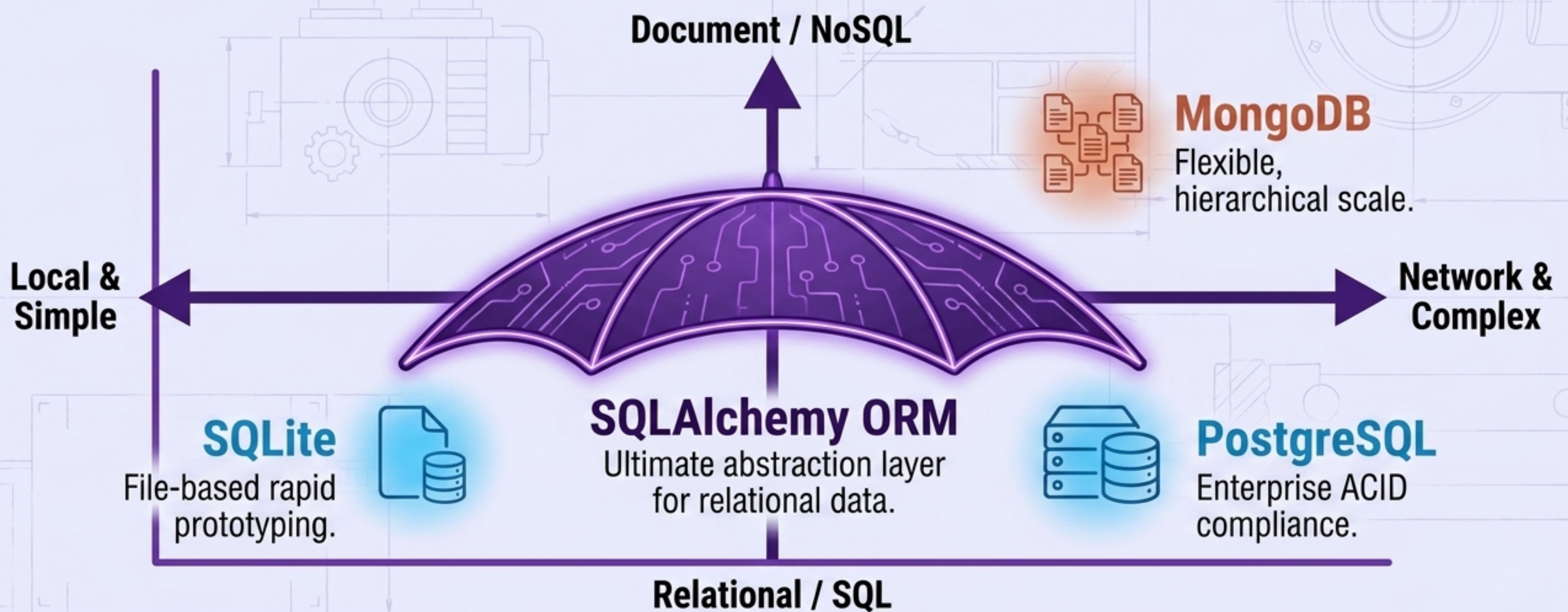
Modelling ORM Relationships

Managing complex data associations intuitively without manual SQL.



- SQLAlchemy abstracts away the need to manually write complex JOIN queries.
- `ForeignKey` establishes strict database-level referential integrity.
- `relationship()` allows fetching associated data effortlessly as a native Python list (e.g., `portfolio.investments`).

The Python Database Ecosystem



All structured query results integrate cleanly into Data Science workflows and Pandas DataFrames, turning persistent data into actionable intelligence.